

CHAPTER 28

finite-difference methods for one-factor models



The aim of this Chapter...

... is to put in place one of the final pieces of the quantitative finance jigsaw. By the end of this chapter you will know one of the most important methods for solving the equations that we have derived in previous chapters.

In this Chapter...

- finite-difference grids
- how to approximate derivatives of a function
- how to go from the Black–Scholes partial differential equation to a *difference* equation
- the explicit finite-difference method, a generalization of the binomial method

28.1 INTRODUCTION

Rarely can we find closed-form solutions for the values of options. Unless the problem is very simple indeed we are going to have to solve a problem numerically. In an earlier chapter I described the binomial method for pricing options. This used the idea of a finite tree structure branching out from the current asset price and the current time right up to the expiry date. One way of thinking of the binomial method is as a method for solving a partial differential equation. Finite-difference methods are no more than a generalization of this concept, although we tend to talk about **grids** and **meshes** rather than 'trees.' Once we have found an equation to solve numerically then it is much easier to use a finite-difference grid than a binomial tree, simply because the transformation from a differential equation (Black–Scholes) to a difference equation is easier when the grid/mesh/tree is nice and regular. Moreover, there are many, many ways the finite-difference method can be improved upon, making it faster and more accurate. The binomial method is not so flexible. And finally, there is a great deal in the mathematical/numerical analysis literature on these and other methods, it would be such a shame to ignore this. The main difference between the binomial method and the finite-difference methods is that the former contains the diffusion, the volatility, in the tree structure. In the finite-difference methods the 'tree' is fixed but parameters change to reflect a changing diffusion.

To those of you who are new to numerical methods let me start by saying that you will find the parabolic partial differential equation very easy to solve numerically. If you are not new to these ideas, but have been brought up on binomial methods, now is the time to wean yourself off them. On a personal note, for solving problems in practice I would say that I use finite-difference methods about 75% of the time, Monte Carlo simulations 20%, and the rest would be explicit formulæ. Those explicit formulæ are almost always just the regular Black–Scholes formulæ for calls and puts, never for barriers for which it is highly dangerous to use constant volatility. Only once have I ever seriously used a binomial method, and that was more to help with modeling than with the numerical analysis.

In this chapter I'm going to show how to approximate derivatives using a grid and then how to write the Black–Scholes equation as a difference equation using this grid. I'll show how this can be done in many ways, discussing the relative merits of each.

When I describe the numerical methods I often use the Black–Scholes equation as the example. *But the methods are all applicable to other problems, such as stochastic interest rates.* I am therefore assuming a certain level of intelligence from my reader, that once you have learned the methods as applied to the equity, currency, commodity worlds you can use them in the fixed-income world. I am sure you won't let me down.

28.2 GRIDS

Figure 28.1 is the binomial tree figure from Chapter 3. This is the structure commonly used for pricing simple non-path-dependent contracts. The idea was explained in Chapter 3. In the world of finite differences we use the grid or mesh shown in Figure 28.2. In the former figure the **nodes** are spaced at equal time intervals and at equal intervals in $\log S$. The finite-difference grid usually has equal **time steps**, the time between nodes, and either equal S steps or equal $\log S$ steps. If we wanted, we could make the grid any shape we wanted.

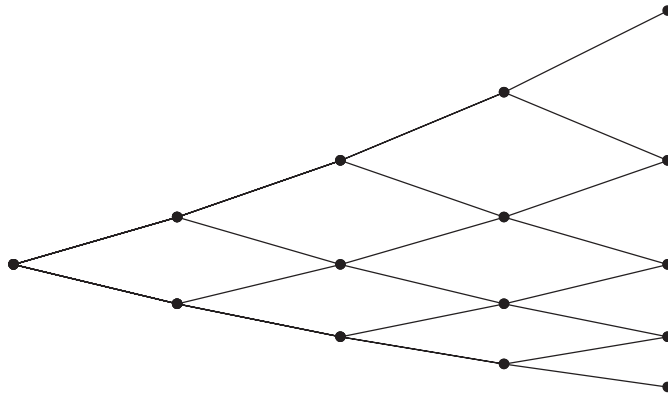


Figure 28.1 The binomial tree.

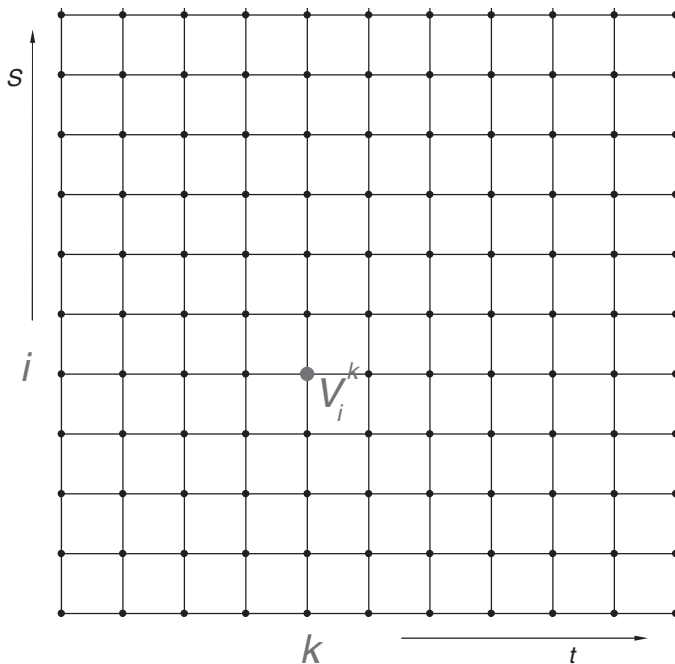


Figure 28.2 The finite-difference grid.

I am only going to describe finite-difference methods with constant time and asset step. There are advantages and disadvantages in this. If we are solving the Black–Scholes equation there is something appealing in having a grid with constant $\log S$ steps, after all, the underlying is following a lognormal random walk. But if you want to use constant $\log S$ steps then it is conceptually simpler to change variables, to write the Black–Scholes equation in terms of the new variable $x = \log S$. Once you have done this then constant $\log S$ step size is equivalent to constant x step size. You could even go so far as to transform to the much neater heat equation as in Chapter 7. One downside to such a

transformation is that equal spacing in $\log S$ means that a lot of grid points are spread around small values of S where there is usually not very much happening. The main reason that I rarely do any transforming of the equation when I am solving it numerically is that I like to solve in terms of the real financial variables since terms of the contract are specified using these real variables: transforming to the heat equation could cause problems for contracts such as barrier options. For other problems such a transformation to something nicer is not even possible. Examples would be an underlying with asset- and time-dependent volatility, or an interest rate product.

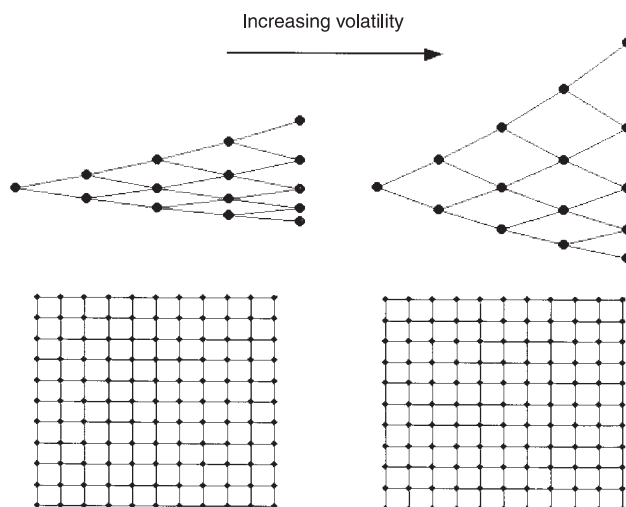
I'm also going to concentrate on backward parabolic equations. Every partial differential equation or numerical analysis book explains methods with reference to the forward equation. But of course, the difference between forward and backward is no more than a change of the sign of the time (but make sure you apply initial conditions to forward equations and final conditions to backward).



Time Out...

Varying volatility

Here's a simple reason why finite differences beat binomial trees. Look at the figures below. They show the effect of increasing vol in a tree and in a grid. As volatility increases the tree spreads out more. The finite-difference mesh stays fixed. This is particularly important when you want to price contracts with important features (such as barriers) at specified asset levels.



28.3 DIFFERENTIATION USING THE GRID

Let's introduce some notation. The time step will be δt and the asset step δS , both of which are constant. Thus the grid is made up of the points at asset values

$$S = i \delta S$$

and times

$$t = T - k \delta t$$

where $0 \leq i \leq I$ and $0 \leq k \leq K$. This means that we will be solving for the asset value going from zero up to the asset value $I \delta S$. Remembering that the Black–Scholes equation is to be solved for $0 \leq S < \infty$ then $I \delta S$ is our approximation to infinity. In practice, this upper bound does not have to be too large. Typically it should be three or four times the value of the exercise price, or more generally, three or four times the value of the asset at which there is some important behavior. In a sense barrier options are easier to solve numerically because you don't need to solve over all values of S ; for an up-and-out option there is no need to make the grid extend beyond the barrier.

I will write the option value at each of these grid points as

$$V_i^k = V(i \delta S, T - k \delta t),$$

so that the superscript is the time variable and the subscript the asset variable. Notice how I've changed the direction of time, as k increases so real time decreases.

Suppose that we know the option value at each of the grid points, can we use this information to find the derivatives of the option value with respect to S and t ? That is, can we find the terms that go into the Black–Scholes equation?

28.4 APPROXIMATING θ

The definition of the first time derivative of V is simply

$$\frac{\partial V}{\partial t} = \lim_{h \rightarrow 0} \frac{V(S, t+h) - V(S, t)}{h}.$$

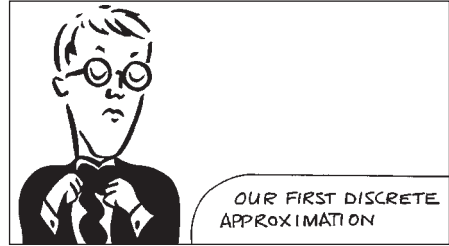
It follows naturally that we can approximate the time derivative from our grid of values using

$$\frac{\partial V}{\partial t}(S, t) \approx \frac{V_i^k - V_i^{k+1}}{\delta t}. \quad (28.1)$$

This is our approximation to the option's theta. It uses the option value at the two points marked in Figure 28.3.

How accurate is this approximation? We can expand the option value at asset value S and time $t - \delta t$ in a Taylor series about the point S, t as follows.

$$V(S, t - \delta t) = V(S, t) - \delta t \frac{\partial V}{\partial t}(S, t) + O(\delta t^2).$$



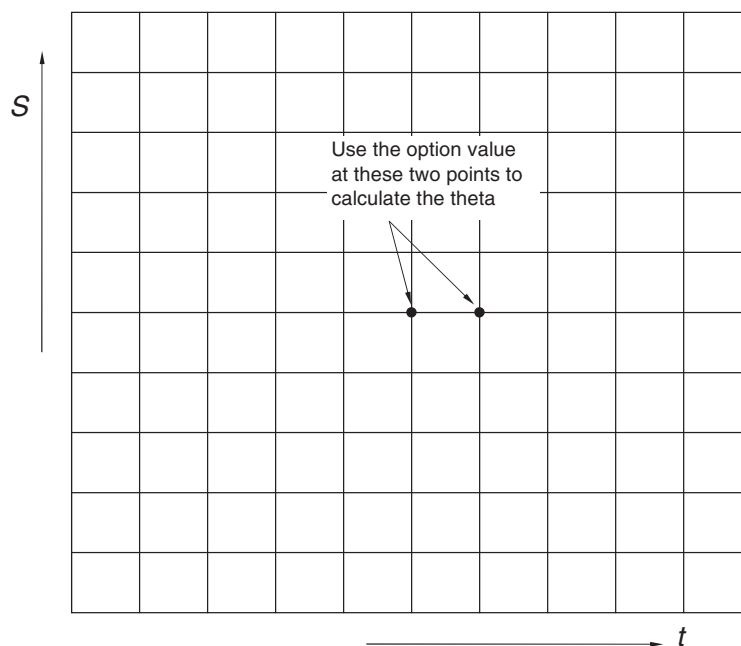


Figure 28.3 An approximation to the theta.

In terms of values at grid points this is just

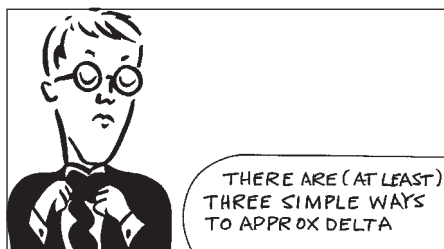
$$V_i^k = V_i^{k+1} + \delta t \frac{\partial V}{\partial t}(S, t) + O(\delta t^2).$$

Which, upon rearranging, is

$$\frac{\partial V}{\partial t}(S, t) = \frac{V_i^k - V_i^{k+1}}{\delta t} + O(\delta t).$$

Our question is answered, the error is $O(\delta t)$. It is possible to be more precise than this, the error depends on the magnitude of the second t derivative. But I won't pursue the details here.

There are other ways of approximating the time derivative of the option value, but this one will do for now.



28.5 APPROXIMATING Δ

The same idea can be used for approximating the first S derivative, the delta. But now I *am* going to present some choices.

Let's examine a cross-section of our grid at one of the time steps. In Figure 28.4 is shown this cross-section. The figure shows three things: the function

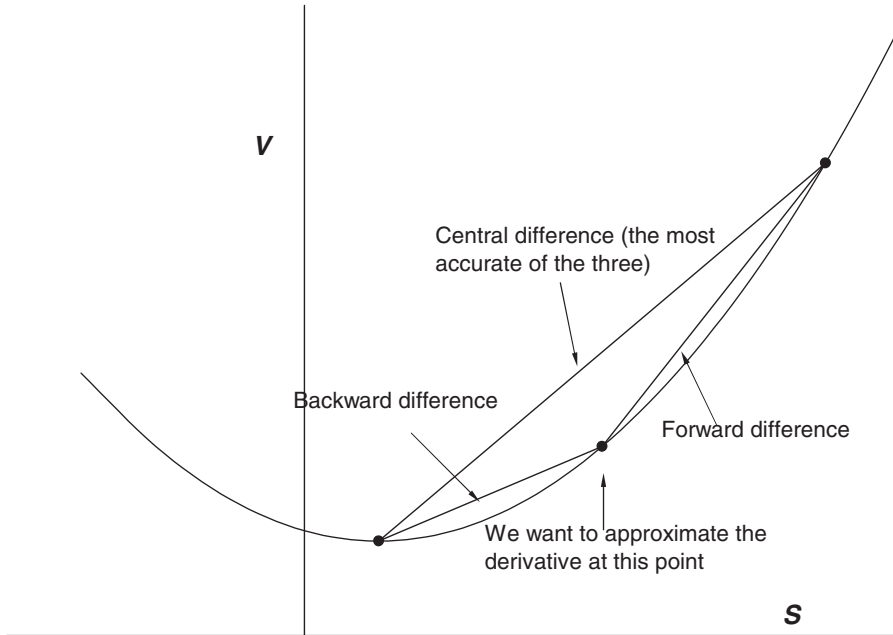


Figure 28.4 Approximations to the delta.

we are approximating (the curve), the values of the function at the grid points (the dots) and three possible approximations to the first derivative (the three straight lines). These three approximations are

$$\frac{V_{i+1}^k - V_i^k}{\delta S}, \quad \frac{V_i^k - V_{i-1}^k}{\delta S} \quad \text{and} \quad \frac{V_{i+1}^k - V_{i-1}^k}{2\delta S}.$$

These are called a **forward difference**, a **backward difference** and a **central difference** respectively.

One of these approximations is better than the others, and it is obvious from the diagram which it is. From a Taylor series expansion of the option value about the point $S + \delta S, t$ we have

$$V(S + \delta S, t) = V(S, t) + \delta S \frac{\partial V}{\partial S}(S, t) + \frac{1}{2} \delta S^2 \frac{\partial^2 V}{\partial S^2}(S, t) + O(\delta S^3).$$

Similarly,

$$V(S - \delta S, t) = V(S, t) - \delta S \frac{\partial V}{\partial S}(S, t) + \frac{1}{2} \delta S^2 \frac{\partial^2 V}{\partial S^2}(S, t) + O(\delta S^3).$$

Subtracting one from the other, dividing by $2\delta S$ and rearranging gives

$$\frac{\partial V}{\partial S}(S, t) = \frac{V_{i+1}^k - V_{i-1}^k}{2\delta S} + O(\delta S^2).$$



Approximations to the delta

The central difference has an error of $O(\delta S^2)$ whereas the error in the forward and backward differences are both much larger, $O(\delta S)$. The central difference is that much more accurate because of the fortunate cancelation of terms, due to the symmetry about S in the definition of the difference.

The central difference calculated at S requires knowledge of the option value at $S + \delta S$ and $S - \delta S$. However, there will be occasions when we do not know one of these values, for example if we are at the extremes of our region, i.e. at $i = 0$ or $i = I$. Then there are times when it may be beneficial to use a one-sided derivative for reasons of stability, an important point which I will come back to. If we do need to use a one-sided derivative, must we use the simple forward or backward difference or is there something better?

The simple forward and backward differences use only two points to calculate the derivative, if we use three points we can get a better order of accuracy. To find the best approximations using three points we need to use Taylor series again.

Suppose I want to use the points S , $S + \delta S$ and $S + 2\delta S$ to calculate the option's delta, how can I do this as accurately as possible? First, expand the option value at the points $S + \delta S$ and $S + 2\delta S$ in a Taylor series:

$$V(S + \delta S, t) = V(S, t) + \delta S \frac{\partial V}{\partial S}(S, t) + \frac{1}{2} \delta S^2 \frac{\partial^2 V}{\partial S^2}(S, t) + O(\delta S^3)$$

and

$$V(S + 2\delta S, t) = V(S, t) + 2\delta S \frac{\partial V}{\partial S}(S, t) + 2\delta S^2 \frac{\partial^2 V}{\partial S^2}(S, t) + O(\delta S^3).$$

If I take the combination

$$-4V(S + \delta S, t) + V(S + 2\delta S, t)$$

I get

$$-3V(S, t) - 2\delta S \frac{\partial V}{\partial S}(S, t) + O(\delta S^3),$$

since the second derivative, $O(\delta S^2)$, terms both cancel. Thus

$$\frac{\partial V}{\partial S}(S, t) = \frac{-3V(S, t) + 4V(S + \delta S, t) - V(S + 2\delta S, t)}{2\delta S} + O(\delta S^2).$$

This approximation is of the same order of accuracy as the central difference, but of better accuracy than the simple forward difference. It uses no information about V for values below S .

If we want to calculate the delta using a better *backward* difference then we would choose

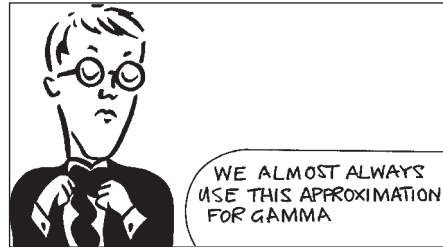
$$\begin{aligned} \frac{\partial V}{\partial S}(S, t) &= \frac{3V(S, t) - 4V(S - \delta S, t) + V(S - 2\delta S, t)}{2\delta S} \\ &\quad + O(\delta S^2). \end{aligned}$$

28.6 APPROXIMATING Γ

The gamma of an option is the second derivative of the option with respect to the underlying. The natural approximation for this is

$$\frac{\partial^2 V}{\partial S^2}(S, t) \approx \frac{V_{i+1}^k - 2V_i^k + V_{i-1}^k}{\delta S^2}.$$

Again, this comes from a Taylor series expansion. The error in this approximation is also $O(\delta S^2)$. I'll leave the demonstration of this as an exercise for the reader.



28.7 EXAMPLE

In Figure 28.5 are shown some option values on the grid. The time step is 0.1 and the asset step is 2.

From these numbers we can estimate the theta as

$$\frac{12 - 13}{0.1} = -10.$$

The delta is approximately

$$\frac{15 - 10}{2 \times 2} = 1.25.$$



Approximating gamma

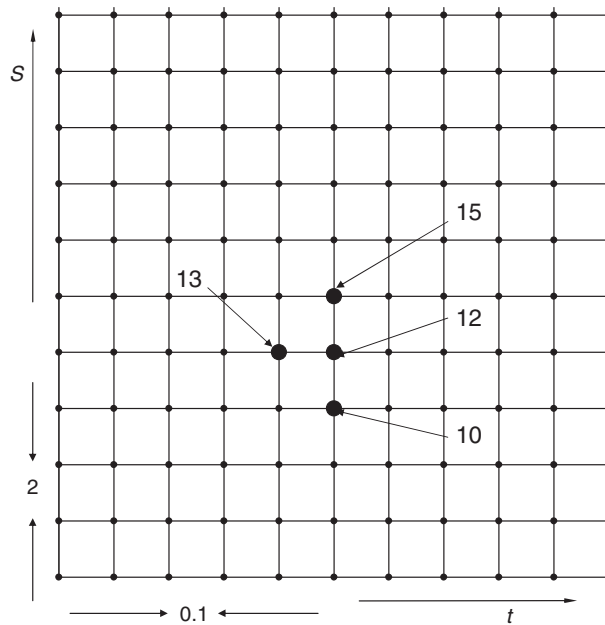


Figure 28.5 Calculation of the greeks.

And the gamma is approximately

$$\frac{15 - 2 \times 12 + 10}{2 \times 2} = 0.25.$$



Time Out...

Some intuition before the math

In terms of the greeks, the Black–Scholes equation can be written

$$\Theta = -\frac{1}{2}\sigma^2 S^2 \Gamma - rS\Delta + rV.$$

We are going to use the equation in this form, with the theta on the left-hand side and the other terms on the right, in the numerical solution. I'll go through all the details shortly, but here's a foretaste.

Imagine we already know the option value as a function of S , at expiry say. In terms of Visual Basic code, call this `VOld(i)`. The distance between asset mesh points is `deltaS`. We can calculate the greeks as follows.

$$\text{Delta} = (\text{VOld}(i + 1) - \text{VOld}(i - 1)) / 2 / \text{deltaS},$$

and

$$\text{Gamma} = (\text{VOld}(i + 1) - 2 * \text{VOld}(i) + \text{VOld}(i - 1)) / \text{deltaS} / \text{deltaS}.$$

So, the Black–Scholes equation gives us

$$\begin{aligned} \text{Theta} = & -0.5 * \text{vol} * \text{vol} * (i * \text{deltaS}) * (i * \text{deltaS}) * \text{Gamma} - \\ & - \text{intrate} * i * \text{deltaS} * \text{Delta} + \text{intrate} * \text{VOld}(i) \end{aligned}$$

We can now step back to find the next value for V at the next time step, call this `VNew(i)`, using

$$\text{VNew}(i) = \text{VOld}(i) - \text{Theta} * \text{deltat}.$$

That's all there is to the explicit finite-difference method. Let's go back to the main thrust of the analysis, but rest assured that the ultimate method is very, very easy.

28.8 **BILINEAR INTERPOLATION**

Suppose that we have an estimate for the option value, or its derivatives, on the mesh points, how can we estimate the value at points in between? The simplest way to do

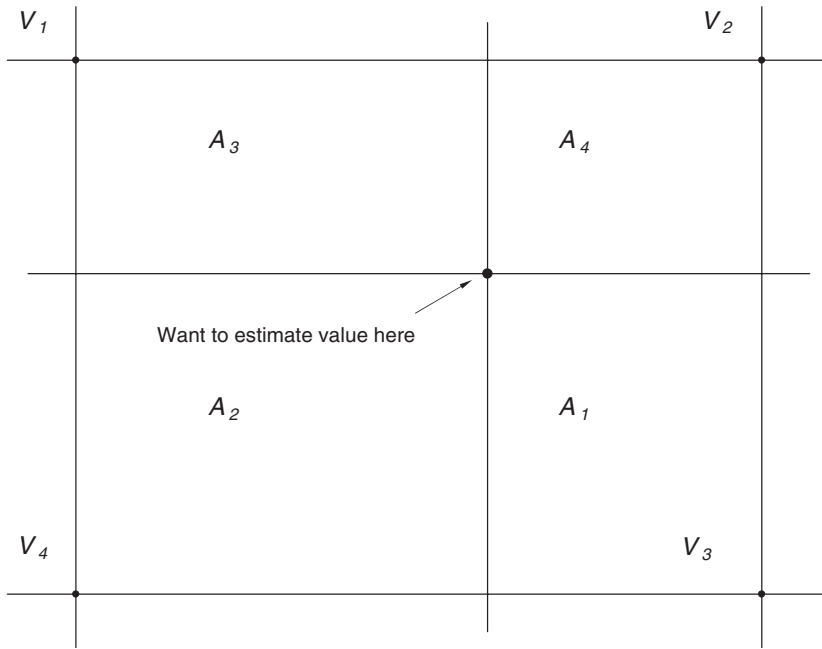


Figure 28.6 Bilinear interpolation.

this is to use a two-dimensional interpolation method called **bilinear interpolation**. This method is most easily explained via the following schematic diagram, Figure 28.6.

We want to estimate the value of the option, say, at the interior point in the figure. The values of the option at the four nearest neighbors are called V_1 , V_2 , V_3 and V_4 , simplifying earlier notation just for this brief section. The areas of the rectangles made by the four corners and the interior point are labeled A_1 , A_2 , A_3 and A_4 . But note that the subscripts for the areas correspond to the subscripts of the option values at the corners *opposite*. The approximation for the option value at the interior point is then

$$\frac{\sum_{i=1}^4 A_i V_i}{\sum_{i=1}^4 A_i}.$$

28.9 FINAL CONDITIONS AND PAYOFFS

We know that at expiry the option value is just the payoff function. This means that we don't have to solve anything for time T . At expiry we have

$$V(S, T) = \text{Payoff}(S)$$

or, in our finite-difference notation,

$$V_i^0 = \text{Payoff}(i \delta S).$$

The right-hand side is a known function. For example, if we are pricing a call option we put

$$V_i^0 = \max(i \delta S - E, 0).$$

This final condition will get our finite-difference scheme started. It will be just like working down the tree in the binomial method.

28.10 BOUNDARY CONDITIONS

When we come to solving the Black–Scholes equation numerically in the next section, we will see that we must specify the option value at the extremes of the region. That is, we must prescribe the option value at $S = 0$ and at $S = I \delta S$. What we specify will depend on the type of option we are solving. I will give some examples.

Example 1 Suppose we want to price a call option. At $S = 0$ we know that the value is always zero, therefore we have

$$V_0^k = 0.$$

Example 2 For large S the call value asymptotes to $S - Ee^{-r(T-t)}$ (plus exponentially small terms). Thus our upper boundary condition could be

$$V_I^k = I \delta S - Ee^{-rk \delta t}.$$

This would be slightly different if we had a dividend.

Example 3 For a put option we have the condition at $S = 0$ that $V = Ee^{-r(T-t)}$. This becomes

$$V_0^k = Ee^{-rk \delta t}.$$

Example 4 The put option becomes worthless for large S and so

$$V_I^k = 0.$$

Example 5 A useful boundary condition to apply at $S = 0$ for most contracts (including calls and puts) is that the diffusion and drift terms ‘switch off.’ This means that on $S = 0$ the payoff is guaranteed, resulting in the condition

$$\frac{\partial V}{\partial t}(0, t) - rV(0, t) = 0.$$

Numerically, this becomes

$$V_0^k = (1 - r \delta t)V_0^{k-1}.$$

Example 6 When the option has a payoff that is at most linear in the underlying for large values of S then you can use the upper boundary condition

$$\frac{\partial^2 V}{\partial S^2}(S, t) \rightarrow 0 \quad \text{as } S \rightarrow \infty.$$

Almost all common contracts have this property. The finite-difference representation is

$$V_I^k = 2V_{I-1}^k - V_{I-2}^k.$$

This is particularly useful because it is independent of the contract being valued meaning that your finite-difference program does not have to be too intelligent.¹

Often there are natural boundaries at finite, non-zero values of the underlying, which means that the domain in which we are solving either does not extend down to zero or up to infinity. Barrier options are the most common form of such contracts.

By way of example, suppose that we want to price an up-and-out call option. This option will be worthless if the underlying ever reaches the value S_u . Clearly,

$$V(S_u, t) = 0.$$

If we are solving this problem numerically how do we incorporate this boundary condition?

The ideal thing to do first of all is to choose an asset step size such that the barrier $S = S_u$ is a grid point, i.e. $S_u/\delta S$ should be an integer. This is to ensure that the boundary condition

$$V_I^k = 0$$

is an accurate representation of the correct boundary condition. Note that we are no longer solving over an asset price range that extends to large S . The upper boundary at $S = S_u$ may be close to the current asset level. In a sense this makes barrier problems easier to solve, the solution region is always smaller than the region over which you would solve a non-barrier problem.

Sometimes it is not possible to make your grid match up with the barrier. This would be the case if the barrier were moving, for example. If this is the case then you are going to have to find an approximation to the boundary condition. There is something that you must not do, and that is to set V equal to zero at the nearest grid point to the barrier. Such an approximation is very inaccurate, of $O(\delta S)$, and will ruin your numerical solution. The trick that we can use to overcome such problems is to introduce a **fictitious point**. This is illustrated in Figure 28.7.

The point $i = I - 1$ is a real point, within the solution region. The point $i = I$ is just beyond the barrier.

Example 7 Suppose that we have the condition that

$$V(S_u, t) = f(t).$$

If we have an ‘out’ option then f would be either zero or the value of the rebate. If we have an ‘in’ option then f is the value of the option into which the barrier option converts.

¹ Sometimes I even use this condition for small values of S , not taking the grid down to $S = 0$.

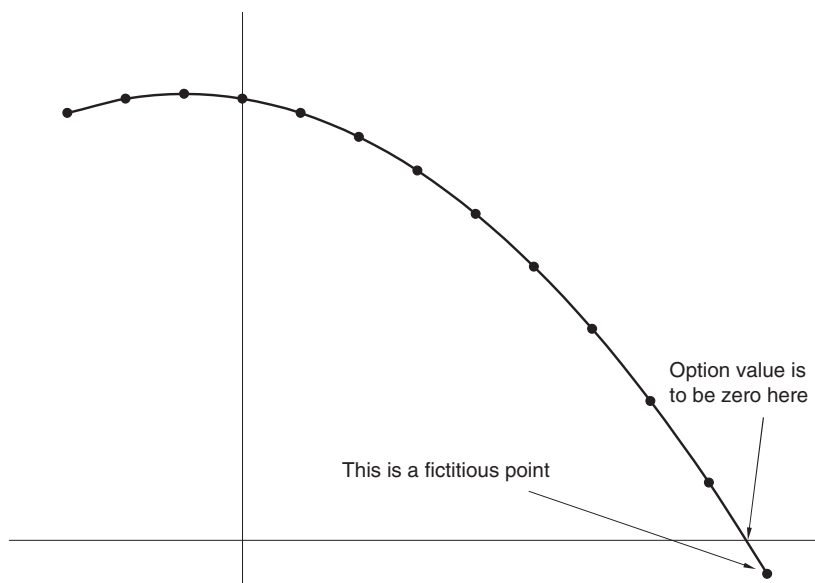


Figure 28.7 A fictitious point, introduced to ensure accuracy in a barrier option boundary condition.

This condition can be approximated by ensuring that the straight line connecting the option values at the two grid points straddling the barrier has the value f at the barrier. Then a good discrete version of this boundary condition is

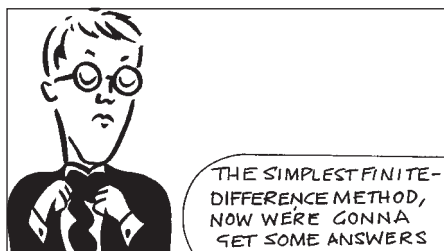
$$V_I^k = \frac{1}{\alpha} (f - (1 - \alpha)V_{I-1}^k)$$

where

$$\alpha = \frac{S_u - (I - 1)\delta S}{\delta S}.$$

This is accurate to $O(\delta S^2)$, the same order of accuracy as in the approximation of the S derivatives.

I have set up all the foundations for us to begin solving some equations. Remember, there has been nothing difficult in what we have done so far, everything is a simple application of Taylor series.



28.11 THE EXPLICIT FINITE-DIFFERENCE METHOD

The Black–Scholes equation is

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0.$$

I'm going to write this as

$$\frac{\partial V}{\partial t} + a(S, t) \frac{\partial^2 V}{\partial S^2} + b(S, t) \frac{\partial V}{\partial S} + c(S, t) V = 0$$

to emphasize the wide applicability of the finite-difference methods. The only constraint we must pose on the coefficients is that if we are solving a backward equation, i.e. imposing final conditions, we must have $a > 0$.

I'm going to take the approximations to the derivatives, explained above, and put them into this equation:

$$\frac{V_i^k - V_i^{k+1}}{\delta t} + a_i^k \left(\frac{V_{i+1}^k - 2V_i^k + V_{i-1}^k}{\delta S^2} \right) + b_i^k \left(\frac{V_{i+1}^k - V_{i-1}^k}{2 \delta S} \right) + c_i^k V_i^k = O(\delta t, \delta S^2).$$

I have used a different line for each of the terms in the original equation.

Points to note:

- The time derivative uses the option values at 'times' k and $k + 1$, whereas the other terms all use values at k .
- The gamma term is a central difference, in practice one never uses anything else.
- The delta term uses a central difference. There are often times when a one-sided derivative is better. We'll see examples later.
- The asset- and time-dependent functions a , b and c have been valued at $S_i = i \delta S$ and $t = T - k \delta t$ with the obvious notation.
- The error in the equation is $O(\delta t, \delta S^2)$.

I am going to rearrange this **difference equation** to put all of the $k + 1$ terms on the left-hand side:

$$V_i^{k+1} = A_i^k V_{i-1}^k + (1 + B_i^k) V_i^k + C_i^k V_{i+1}^k \quad (28.2)$$

where

$$\begin{aligned} A_i^k &= v_1 a_i^k - \frac{1}{2} v_2 b_i^k, \\ B_i^k &= -2v_1 a_i^k + \delta t c_i^k \end{aligned}$$

and

$$C_i^k = v_1 a_i^k + \frac{1}{2} v_2 b_i^k$$

where

$$v_1 = \frac{\delta t}{\delta S^2} \quad \text{and} \quad v_2 = \frac{\delta t}{\delta S}.$$

The error in this is $O(\delta t^2, \delta t \delta S^2)$, I will come back to this in a moment. The error in the approximation of the differential equation is called the **local truncation error**.

Equation (28.2) only holds for $i = 1, \dots, I - 1$, i.e. for interior points, since V_{-1}^k and V_{I+1}^k are not defined. Thus there are $I - 1$ equations for the $I + 1$ unknowns, the V_i^k . The remaining two equations come from the two boundary conditions on $i = 0$ and $i = I$. These two end points are treated separately.

If we know V_i^k for all i then Equation (28.2) tells us V_i^{k+1} . Since we know V_i^0 , the payoff function, we can easily calculate V_i^1 , which is the option value one time step

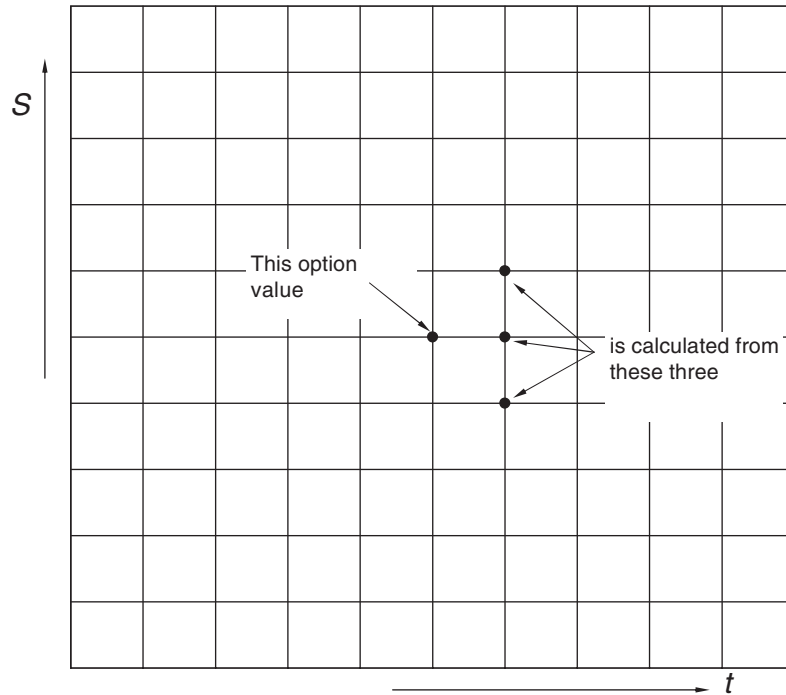


Figure 28.8 The relationship between option values in the explicit method.

before expiry. Using these values we can work step by step back down the grid as far as we want. Because the relationship between the option values at time step $k + 1$ is a simple function of the option values at time step k this method is called the **explicit finite-difference method**. The relationship between the option values in Equation (28.2) is shown in Figure 28.8.

Equation (28.2) is only used to calculate the option value for $1 \leq i \leq I - 1$ since the equation requires knowledge of the option values at $i - 1$ and $i + 1$. This is where the boundary conditions come in. Typically we either have a V_i^k being prescribed at $i = 0$ and $i = I$ or, as suggested above, we might prescribe a relationship between the option value at an end point and interior values. This idea is illustrated in the following Visual Basic code fragment. This code fragment does not have all the declarations, etc. at the top, nor the return of any answers. I will give a full function shortly, for the moment I want you to concentrate on setting up the final condition and the finite-difference time loop.

The array $V(i, k)$ holds the option values. Unless we wanted to store all option values for all time steps this would not be the most efficient way of writing the program, I will describe a better way in a moment.

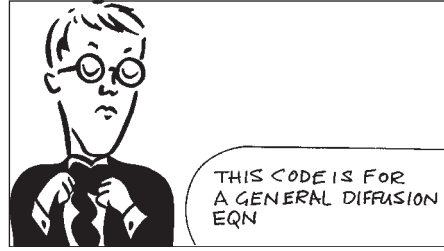
First, set up the final condition, the payoff.

```
For i = 0 To NoAssetSteps
    S(i) = i * AssetStep
    V(i, 0) = CallPayoff(S(i)) ' Set up final condition
Next i
```


Now we can work backwards in time using the following time loop.

```
' Time loop
For k = 1 To NoTime steps
    RealTime = Expiry - k * Time step
    For i = 1 To NoAssetSteps - 1
        V(i, k + 1) = A(S(i), RealTime) _
            * V(i - 1, k) + _
            B(S(i), RealTime) _
            * V(i, k) + _
            C(S(i), RealTime) _
            * V(i + 1, k)

    Next i
' BC at S=0
    V(0, k + 1) = 0
' BC at "infinity"
    V(NoAssetSteps, k + 1) = 2 * V(NoAssetSteps - 1, k + 1) _
        - V(NoAssetSteps - 2, k + 1)
Next k
```



The explicit finite-difference algorithm is simple, the functions $A(S(i), \text{RealTime})$, $B(S(i), \text{RealTime})$ and $C(S(i), \text{RealTime})$ are defined elsewhere and are in terms of the asset price $S(i)$ and the time RealTime . Since I am valuing a call option here the boundary condition at $S = 0$ is simply $V(0, k + 1) = 0$ but the boundary condition I have used at the upper boundary $i = \text{NoAssetSteps}$ is that the gamma is zero.

28.11.1 The Black–Scholes equation

For the Black–Scholes equation with dividends the coefficients above simplify to

$$A_i^k = \frac{1}{2} \left(\sigma^2 i^2 - (r - D)i \right) \delta t,$$

$$B_i^k = - \left(\sigma^2 i^2 + r \right) \delta t$$

and

$$C_i^k = \frac{1}{2} \left(\sigma^2 i^2 + (r - D)i \right) \delta t.$$

This uses $S = i \delta S$. If the volatility, interest rate and dividend are constant then there is no time or k dependence in these coefficients.

28.11.2 Convergence of the explicit method

I can write the value of the option at any i point at the final time step K as

$$V_i^K = V_i^0 + \sum_{k=0}^{K-1} (V_i^{k+1} - V_i^k).$$

Each of the terms in this summation is in error by $O(\delta t^2, \delta t \delta S^2)$. This means that the error in the final option value is

$$O(K\delta t^2, K\delta t \delta S^2)$$

since there are K terms in the summation. If we value the option at a finite value of T then $K = O(\delta t^{-1})$ so that the error in the final option value is $O(\delta t, \delta S^2)$.

Although the explicit method is simple to implement it does not always converge. Convergence of the method depends on the size on the time step, the size of the asset step and the size of the coefficients a , b and c .

The technique often used to demonstrate convergence is quite fun, so I'll show you how it is done. The method, as I describe it, is not rigorous but it can be made so.

Ask the question, 'If a small error is introduced into the solution, is it magnified by the numerical method or does it decay away?' If a small error, introduced by rounding errors for example, becomes a large error then the method is useless. The usual way to analyze such stability is to look for a solution of the equation, Equation (28.2), of the form²

$$V_i^k = \alpha^k e^{2\pi i \sqrt{-1}/\lambda}. \quad (28.3)$$

In other words, I'm going to look for an oscillatory solution with a wavelength of λ . If I find that $|\alpha| > 1$ then there is instability. Note that I am not worried about how the oscillation gets started, I could interpret this special solution as part of a Fourier series analysis.

Substituting (28.3) into (28.2) I get

$$\alpha = \left(1 + c_i^k \delta t + 2a_i^k v_1 (\cos(2\pi/\lambda) - 1)\right) + \sqrt{-1} b_i^k v_2 \sin(2\pi/\lambda).$$

It turns out that to have $|\alpha| < 1$, for stability, we require

$$c_i^k \leq 0,$$

$$2v_1 a_i^k - \delta t c_i^k \leq 1$$

and

$$\frac{1}{2} v_2 |b_i^k| \leq v_1 a_i^k.$$

To get this result, I have assumed that all the coefficients are slowly varying over the δS length scales.

In financial problems we almost always have a negative c , often it is simply $-r$ where r is the risk-free interest rate. The other two constraints are what really limit the applicability of the explicit method.

Typically we choose v_1 to be $O(1)$ in which case the second constraint is approximately

$$v_1 \leq \frac{1}{2a}$$

(ignoring sub- and superscripts on a). This is a serious limitation on the size of the time step,

$$\delta t \leq \frac{\delta S^2}{2a}.$$

² This is the only place in the book that I use $\sqrt{-1}$. Normally I'd write this as i but I need i for other quantities.

If we want to improve accuracy by halving the asset step, say, we must reduce the time step by a factor of four. The computation time then goes up by a factor of *eight*. The improvement in accuracy we would get from such a reduction in step sizes is a factor of four since the explicit finite-difference method is accurate to $O(\delta t, \delta S^2)$.

In the Black–Scholes equation this time step constraint becomes

$$\delta t \leq \frac{\delta S^2}{2a} = \frac{\delta S^2}{\sigma^2 S^2} = \frac{1}{\sigma^2} \left(\frac{\delta S}{S} \right)^2.$$

This constraint depends on the asset price. Since δt will be independent of S the constraint is most restrictive for the largest S in the grid. If there are I equally spaced asset grid points then the constraint is simply

$$\delta t \leq \frac{1}{\sigma^2 I^2}.$$

If the time step constraint is not satisfied, if it is too large, then the instability is obvious from the results. The instability is so serious, and so oscillatory, that it is easily noticed. It is unlikely that you will get a false but believable result if you use the explicit method.

The final constraint can also be a serious restriction. It can be written as

$$\delta S \leq \frac{2a}{|b|}. \quad (28.4)$$

If we are solving the Black–Scholes equation this restriction does not make much difference in practice unless the volatility is very small. It can be important in other problems though and I will show how to get past this restriction after the next Visual Basic programs.

28.12 THE CODE #1: EUROPEAN OPTION

The following VB code will output the value of a call or put option (depending on whether PType is C or P). The time step is hard coded and depends on the asset step. It has been chosen to be just about as big as possible while still satisfying the constraint on v_1 , so that the scheme shouldn't be unstable.

A whole array of option value versus stock price and time is output in this example.

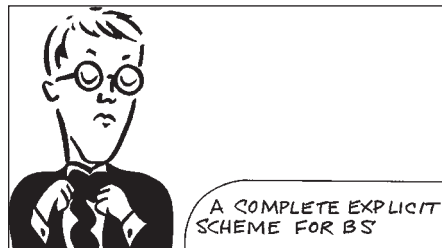
```
Function Option_Value_3D(Vol, Int_Rate, _
    PType, Strike, Expiration, NAS)
' NAS is number of asset steps

ReDim S(0 To NAS) As Double ' Asset array

dS = 2 * Strike / NAS ' 'Infinity' is _
    twice the strike
dt = 0.9 / Vol ^ 2 / NAS ^ 2 ' For _
    stability
NTS = Int(Expiration / dt) + 1 ' Number _
    of time steps
```



Explicit
finite-difference
code



```

dt = Expiration / NTS ' To ensure that expiration is an integer number _
    of time steps away

ReDim V(0 To NAS, 0 To NTS) As Double ' Option value array

q = 1
If PType = "P" Then q = -1 ' Test for call or put

For i = 0 To NAS
    S(i) = i * dS ' Set up S array
    V(i, 0) = Application.Max(q * (S(i) - Strike), 0) ' Set up payoff
Next i

For k = 1 To NTS ' Time loop
    For i = 1 To NAS - 1 ' Asset loop. End points treated separately
        Delta = (V(i + 1, k - 1) - V(i - 1, k - 1)) / 2 / dS ' Central difference
        Gamma = (V(i + 1, k - 1) - 2 * V(i, k - 1) + V(i - 1, k - 1)) / dS / dS _
            ' Central difference
        Theta = -0.5 * Vol ^ 2 * S(i) ^ 2 * Gamma - _
            Int_Rate * S(i) * Delta + Int_Rate * _
            V(i, k - 1) ' Black-Scholes
        V(i, k) = V(i, k - 1) - dt * Theta
    Next i

    V(0, k) = V(0, k - 1) * (1 - Int_Rate * dt) ' Boundary condition at S=0
    V(NAS, k) = 2 * V(NAS - 1, k) - V(NAS - 2, k) ' Boundary condition at _
        S=infinity

Next k

Option_Value_3D = V ' Output array

End Function

```

Results from this program for $\sigma = 0.2$, $r = 0.05$, $E = 100$, $T = 1$ and $NAS = 20$ are shown in Table 28.1 and Figure 28.9 for a call option.

Table 28.1 Call option values output by the explicit finite-difference code. Stock price ranges from 60 to 100, time from 0 (expiration) to 1.

	0.000	0.111	0.222	0.333	0.444	0.556	0.667	0.778	0.889	1.000
60	0.000	0.000	0.000	0.000	0.001	0.004	0.011	0.023	0.041	0.066
70	0.000	0.000	0.001	0.008	0.028	0.067	0.126	0.207	0.308	0.430
80	0.000	0.000	0.037	0.141	0.310	0.534	0.799	1.096	1.416	1.754
90	0.000	0.128	0.592	1.182	1.812	2.450	3.080	3.700	4.306	4.899
100	0.000	2.253	3.819	5.054	6.109	7.054	7.925	8.741	9.515	10.255
110	10.000	10.671	11.587	12.535	13.455	14.337	15.180	15.990	16.770	17.523
120	20.000	20.555	21.159	21.826	22.529	23.247	23.967	24.683	25.391	26.089
130	30.000	30.555	31.109	31.680	32.272	32.882	33.504	34.134	34.768	35.402
140	40.000	40.555	41.106	41.658	42.213	42.777	43.348	43.926	44.509	45.095

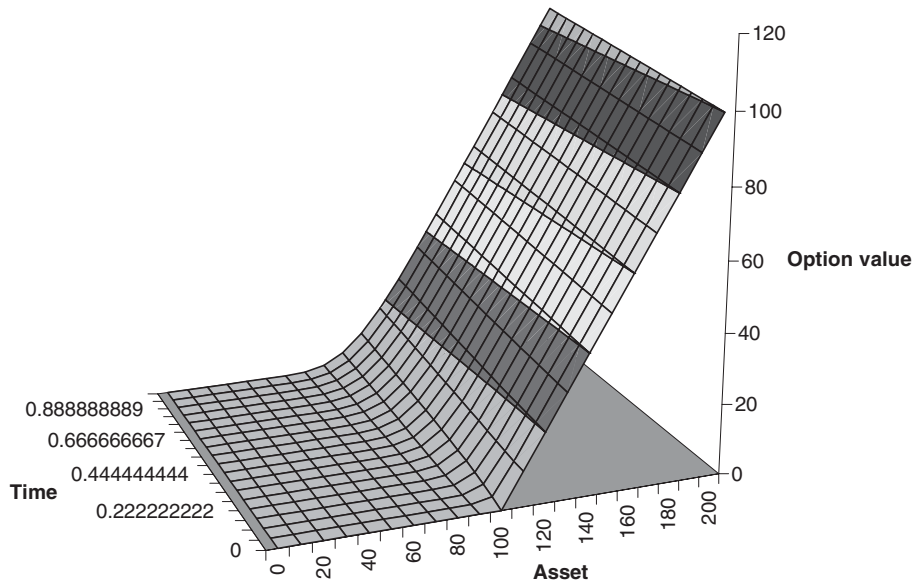


Figure 28.9 Call option values output by the explicit finite-difference code.

Table 28.2 and Figure 28.10 show the results for a put, same parameter values.

The error between the results of this explicit finite-difference program and the exact Black–Scholes formula as a function of the underlying is shown in Figure 28.11 for 50 asset points, a volatility of 20%, and interest rate of 10% an expiry of one year and a strike of 100. The option is a call.

The logarithm of the absolute error as a function of the logarithm of the square of the asset step size is shown in Figure 28.12. The $O(\delta S^2)$ behavior is very obvious. In these calculations I have kept ν_1 constant.

In Figure 28.13 is shown the error as a function of calculation time; time units will depend on your machine.

Table 28.2 Put option values output by the explicit finite-difference code. Stock price ranges from 60 to 100, time from 0 (expiration) to 1.

	0.000	0.111	0.222	0.333	0.444	0.556	0.667	0.778	0.889	1.000
60	40.000	39.445	38.894	38.345	37.801	37.261	36.728	36.204	35.688	35.182
70	30.000	29.445	28.894	28.353	27.827	27.323	26.843	26.387	25.955	25.546
80	20.000	19.445	18.930	18.486	18.110	17.790	17.516	17.276	17.063	16.871
90	10.000	9.573	9.486	9.527	9.612	9.706	9.798	9.880	9.953	10.015
100	0.000	1.699	2.713	3.399	3.909	4.311	4.642	4.921	5.162	5.372
110	0.000	0.116	0.481	0.880	1.254	1.593	1.898	2.171	2.417	2.639
120	0.000	0.000	0.052	0.171	0.328	0.503	0.684	0.864	1.038	1.205
130	0.000	0.000	0.003	0.025	0.071	0.138	0.221	0.315	0.415	0.518
140	0.000	0.000	0.000	0.002	0.013	0.033	0.065	0.106	0.156	0.211

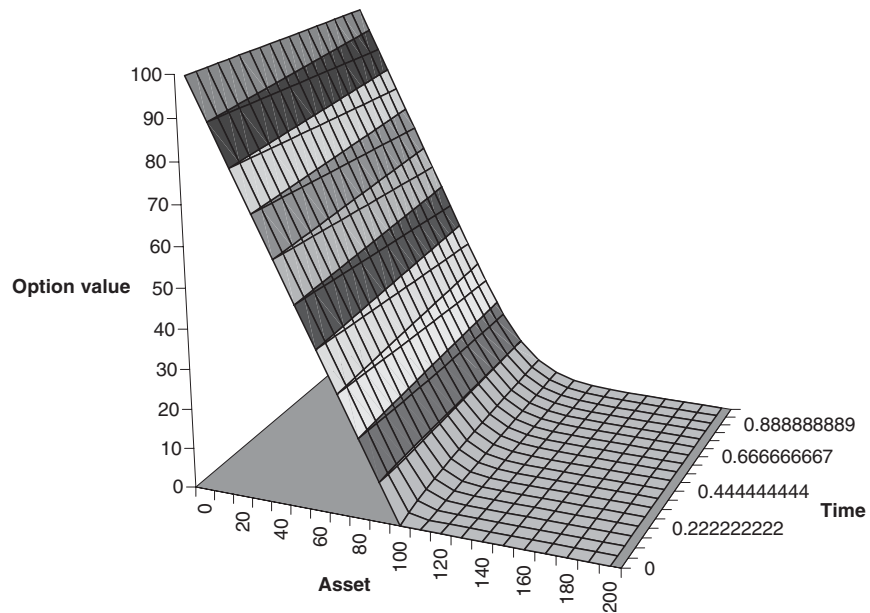


Figure 28.10 Put option values output by the explicit finite-difference code.

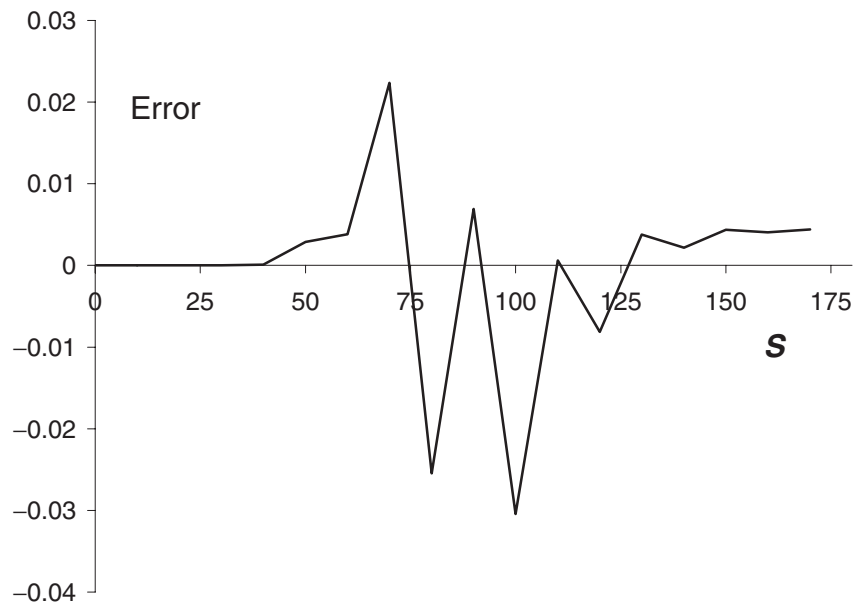


Figure 28.11 Error as a function of the underlying using the finite-difference scheme.

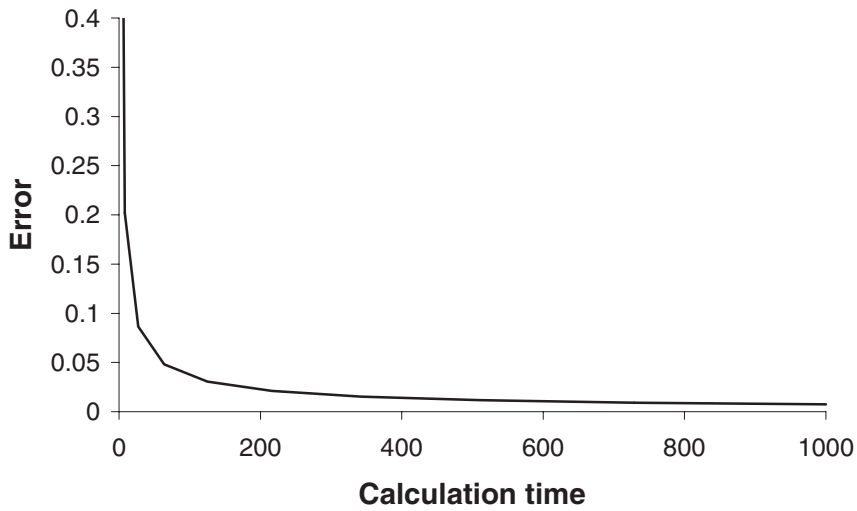


Figure 28.12 $\text{Log}(\text{Error})$ as a function of $\text{log}(\delta S^2)$ using the finite-difference scheme.

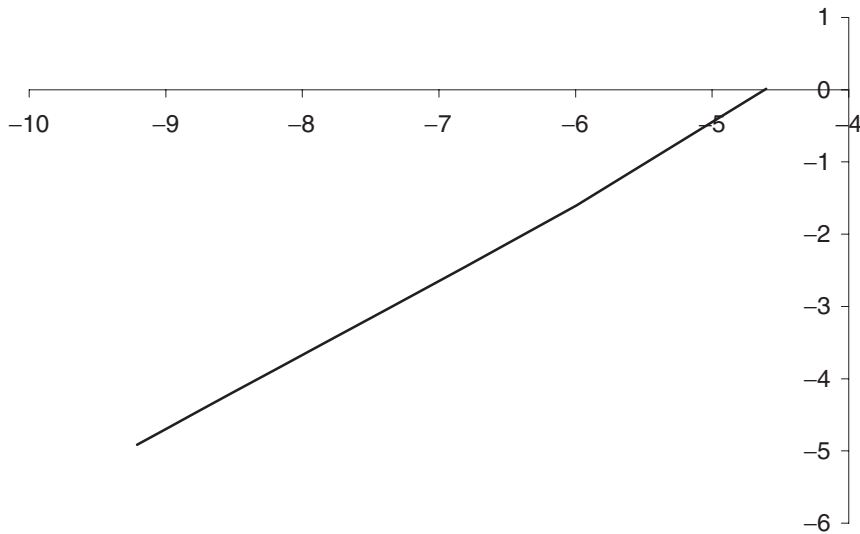


Figure 28.13 Absolute error as a function of the calculation time using the finite-difference scheme.

28.13 THE CODE #2: AMERICAN EXERCISE

The code changes trivially when we have early exercise.

```
Function Option_Value_3D_US(Vol, Int_Rate, PType, Strike, Expiration, _
                           EType, NAS)
' NAS is number of asset steps
```

```

ReDim S(0 To NAS) As Double ' Asset array
ReDim Payoff(0 To NAS) As Double ' Payoff array

dS = 2 * Strike / NAS ' 'Infinity' is twice the strike
dt = 0.9 / Vol ^ 2 / NAS ^ 2 ' For stability
NTS = Int(Expiration / dt) + 1 ' Number of time steps
dt = Expiration / NTS ' To ensure that expiration is an integer number _
    of time steps away

ReDim V(0 To NAS, 0 To NTS) As Double ' Option value array

q = 1
If PType = "P" Then q = -1 ' Test for call or put

For i = 0 To NAS
    S(i) = i * dS ' Set up S array
    V(i, 0) = Application.Max(q * (S(i) - Strike), 0) ' Set up payoff
    Payoff(i) = V(i, 0) ' Store payoff
Next i

For k = 1 To NTS ' Time loop
    For i = 1 To NAS - 1 ' Asset loop. End points treated separately
        Delta = (V(i + 1, k - 1) - V(i - 1, k - 1)) / 2 / dS ' Central difference
        Gamma = (V(i + 1, k - 1) - 2 * V(i, k - 1) + V(i - 1, k - 1)) / dS / _
            dS ' Central difference
        Theta = -0.5 * Vol ^ 2 * S(i) ^ 2 * Gamma - _
            Int_Rate * S(i) * Delta + Int_Rate * _
            V(i, k - 1) ' Black-Scholes
        V(i, k) = V(i, k - 1) - dt * Theta
    Next i

    V(0, k) = V(0, k - 1) * (1 - Int_Rate * dt) ' Boundary condition at S=0
    V(NAS, k) = 2 * V(NAS - 1, k) - V(NAS - 2, k) ' Boundary condition _
        at S=infinity

    If EType = "Y" Then ' Check for early exercise
        For i = 0 To NAS
            V(i, k) = Application.Max(V(i, k), Payoff(i))
        Next i
    End If

Next k

Option_Value_3D_US = V ' Output array

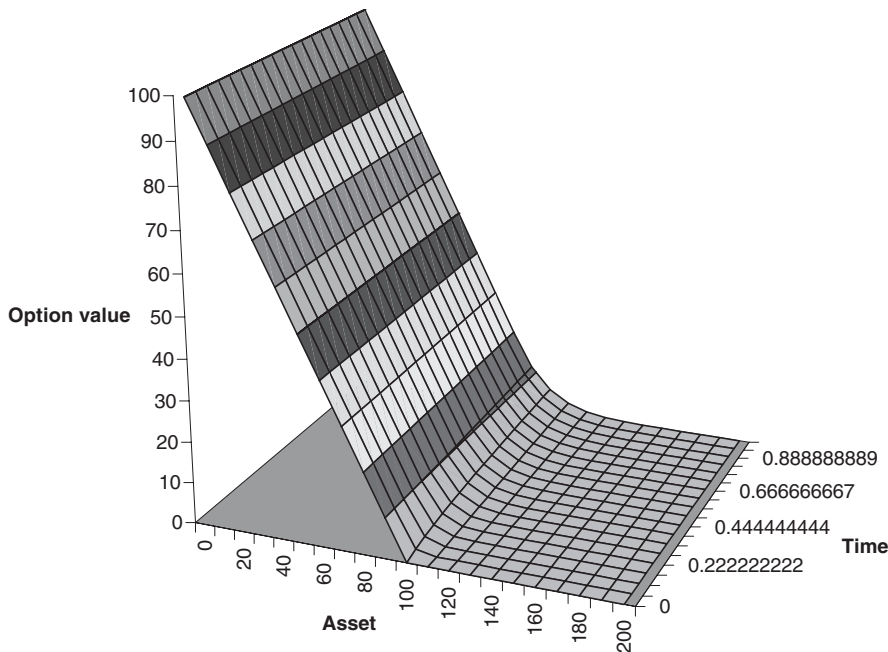
End Function

```

In Table 28.3 and Figure 28.14 are results for the American put, same parameters as above.

Table 28.3 American put option values output by the explicit finite-difference code. Stock price ranges from 60 to 100, time from 0 (expiration) to 1.

	0.000	0.111	0.222	0.333	0.444	0.556	0.667	0.778	0.889	1.000
60	40.000	40.000	40.000	40.000	40.000	40.000	40.000	40.000	40.000	40.000
70	30.000	30.000	30.000	30.000	30.000	30.000	30.000	30.000	30.000	30.000
80	20.000	20.000	20.000	20.000	20.000	20.000	20.000	20.000	20.000	20.000
90	10.000	10.000	10.000	10.060	10.226	10.434	10.653	10.869	11.074	11.267
100	0.000	1.726	2.810	3.549	4.102	4.551	4.935	5.275	5.581	5.859
110	0.000	0.116	0.491	0.909	1.305	1.666	1.994	2.294	2.571	2.827
120	0.000	0.000	0.053	0.174	0.338	0.523	0.714	0.905	1.093	1.276
130	0.000	0.000	0.003	0.025	0.073	0.143	0.230	0.328	0.434	0.544
140	0.000	0.000	0.000	0.002	0.013	0.034	0.067	0.110	0.162	0.221

**Figure 28.14** American put option values output by the explicit finite-difference code.

28.14 THE CODE #3: 2-D OUTPUT

The above code is fine if you want to output the entire function $V(S, t)$. But if you only want to know today's curve or value it is very inefficient since it stores too much information. The following code only uses `Vold` and `VNew` to store enough information to calculate the curve today. The code also outputs the greeks.

```
Function Option_Value_2D_US(Vol, Int_Rate, PType, Strike, Expiration, _
                           EType, NAS)
' NAS is number of asset steps
```

```

ReDim S(0 To NAS) As Double ' Asset array
ReDim VOld(0 To NAS) As Double ' One option array
ReDim VNew(0 To NAS) As Double ' Second option array
ReDim Payoff(0 To NAS) As Double ' Payoff array
ReDim Dummy(0 To NAS, 1 To 6) As Double ' Used for storing data for _
    output

dS = 2 * Strike / NAS ' 'Infinity' is twice the strike
dt = 0.9 / Vol ^ 2 / NAS ^ 2 ' For stability
NTS = Int(Expiration / dt) + 1 ' Number of time steps
dt = Expiration / NTS ' To ensure that expiration is an integer number _
    of time steps away

q = 1
If PType = "P" Then q = -1 ' Test for call or put

For i = 0 To NAS
    S(i) = i * dS ' Set up S array
    VOld(i) = Application.Max(q * (S(i) - Strike), 0) ' Set up payoff
    Payoff(i) = VOld(i) ' Store payoff
    Dummy(i, 1) = S(i) ' First column of Dummy is the stock price
    Dummy(i, 2) = Payoff(i) ' Second column of Dummy is the payoff
Next i

For k = 1 To NTS ' Time loop
    For i = 1 To NAS - 1 ' Asset loop. End points treated separately
        Delta = (VOld(i + 1) - VOld(i - 1)) / 2 / dS ' Central difference
        Gamma = (VOld(i + 1) - 2 * VOld(i) + VOld(i - 1)) / dS / dS ' Central _
            difference
        Theta = -0.5 * Vol ^ 2 * S(i) ^ 2 * Gamma - _
            Int_Rate * S(i) * Delta + Int_Rate * _
            VOld(i) ' Black-Scholes
        VNew(i) = VOld(i) - dt * Theta ' Update option value
    Next i

    VNew(0) = VOld(0) * (1 - Int_Rate * dt) ' Boundary condition at S=0
    VNew(NAS) = 2 * VNew(NAS - 1) - VNew(NAS - 2) ' Boundary condition at _
        S=infinity

    For i = 0 To NAS ' Replace Old with New
        VOld(i) = VNew(i)
    Next i

    If EType = "Y" Then ' Check for early exercise
        For i = 0 To NAS
            VOld(i) = Application.Max(VOld(i), Payoff(i))
        Next i
    End If

Next k

```

```

For i = 1 To NAS - 1
  Dummy(i, 3) = VOld(i) ' Third column of Dummy is option value
  Dummy(i, 4) = (VOld(i + 1) - VOld(i - 1)) / 2 / dS ' Fourth column of _
    Dummy is delta
  Dummy(i, 5) = (VOld(i + 1) - 2 * VOld(i) + VOld(i - 1)) / dS / dS
    ' Fifth column of Dummy is gamma
  Dummy(i, 6) = -0.5 * Vol ^ 2 * S(i) ^ 2 * Dummy(i, 5) - _
    Int_Rate * S(i) * Dummy(i, 4) + Int_Rate * VOld(i) ' Sixth _
    column of Dummy is theta
Next i

Dummy(0, 3) = VOld(0)
Dummy(NAS, 3) = VOld(NAS)
Dummy(0, 4) = (VOld(1) - VOld(0)) / dS ' End points need to be treated _
  separately for delta, gamma and theta
Dummy(NAS, 4) = (VOld(NAS) - VOld(NAS - 1)) / dS
Dummy(0, 5) = 0
Dummy(NAS, 5) = 0
Dummy(0, 6) = Int_Rate * VOld(0)
Dummy(NAS, 6) = -0.5 * Vol ^ 2 * S(NAS) ^ 2 * Dummy(NAS, 5) - _
  Int_Rate * S(NAS) * Dummy(NAS, 4) + Int_Rate * VOld(NAS)

Option_Value_2D_US = Dummy ' Output array

End Function

```

Table 28.4 shows the results from this code, same parameters but now 40 asset steps.

28.15 UPWIND DIFFERENCING

The constraint (28.4) can be avoided if we use a one-sided difference instead of a central difference for the first derivative of the option value with respect to the asset. As I said in Chapter 7 the first S derivative represents a drift term. This drift has a direction associated with it, as t decreases, moving away from expiry, so the drift is towards smaller S . In a sense, this makes



the forward price of the asset a better variable to use. Anyway, the numerical scheme can make use of this by using a one-sided difference. That is the situation for the Black–Scholes equation. More generally, the approximation that we use for delta in the equation could depend on the sign of b . For example, use the following

$$\text{if } b(S, t) \geq 0 \text{ then } \frac{\partial V}{\partial S}(S, t) = \frac{V_{i+1}^k - V_i^k}{\delta S}$$

but if

$$b(S, t) < 0 \text{ then } \frac{\partial V}{\partial S}(S, t) = \frac{V_i^k - V_{i-1}^k}{\delta S}.$$

Table 28.4 Call option values and greeks output by the explicit finite-difference code.

Stock	Payoff	Option	Delta	Gamma	Theta
0	0.000	0.000	0.000	0.000	0.000
5	0.000	0.000	0.000	0.000	0.000
10	0.000	0.000	0.000	0.000	0.000
15	0.000	0.000	0.000	0.000	0.000
20	0.000	0.000	0.000	0.000	0.000
25	0.000	0.000	0.000	0.000	0.000
30	0.000	0.000	0.000	0.000	0.000
35	0.000	0.000	0.000	0.000	0.000
40	0.000	0.000	0.000	0.000	-0.001
45	0.000	0.001	0.000	0.000	-0.005
50	0.000	0.003	0.002	0.000	-0.023
55	0.000	0.016	0.005	0.001	-0.086
60	0.000	0.058	0.016	0.003	-0.256
65	0.000	0.173	0.038	0.006	-0.617
70	0.000	0.437	0.078	0.010	-1.235
75	0.000	0.953	0.139	0.015	-2.113
80	0.000	1.832	0.222	0.019	-3.168
85	0.000	3.174	0.321	0.021	-4.255
90	0.000	5.044	0.429	0.022	-5.224
95	0.000	7.461	0.536	0.021	-5.962
100	0.000	10.403	0.636	0.019	-6.425
105	5.000	13.816	0.723	0.016	-6.625
110	10.000	17.628	0.795	0.013	-6.614
115	15.000	21.764	0.852	0.010	-6.459
120	20.000	26.149	0.896	0.007	-6.225
125	25.000	30.722	0.928	0.005	-5.965
130	30.000	35.430	0.951	0.004	-5.712
135	35.000	40.235	0.968	0.003	-5.488
140	40.000	45.107	0.979	0.002	-5.301
145	45.000	50.023	0.986	0.001	-5.153
150	50.000	54.969	0.991	0.001	-5.039
155	55.000	59.935	0.994	0.001	-4.954
160	60.000	64.913	0.997	0.000	-4.892
165	65.000	69.900	0.998	0.000	-4.848
170	70.000	74.892	0.999	0.000	-4.817
175	75.000	79.886	0.999	0.000	-4.796
180	80.000	84.883	0.999	0.000	-4.781
185	85.000	89.881	1.000	0.000	-4.770
190	90.000	94.880	1.000	0.000	-4.761
195	95.000	99.878	1.000	0.000	-4.754
200	100.000	104.877	1.000	0.000	-4.754

This removes the limitation (28.4) on the asset step size, improving stability. However, since these one-sided differences are only accurate to $O(\delta S)$ the numerical method is less accurate.

The use of one-sided differences that depend on the sign of the coefficient b is called **upwind differencing**.³ There is a small refinement of the technique in the choice of the value chosen for the function b :

$$\text{if } b(S, t) \geq 0 \text{ then } b(S, t) \frac{\partial V}{\partial S}(S, t) = b_{i+\frac{1}{2}}^k \frac{V_{i+1}^k - V_i^k}{\delta S}$$

but if

$$b(S, t) < 0 \text{ then } b(S, t) \frac{\partial V}{\partial S}(S, t) = b_{i-\frac{1}{2}}^k \frac{V_i^k - V_{i-1}^k}{\delta S}.$$

Notice how I have used the midpoint value for b .⁴

Below is a Visual Basic code fragment that uses a one-sided difference, which one depending on the sign of the drift term. This code fragment can be used for interest rate products, since it is solving

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma(r)^2 \frac{\partial^2 V}{\partial r^2} + (\mu(r) - \lambda(r)\sigma(r)) \frac{\partial V}{\partial r} - rV = 0.$$

Note the arbitrary $\sigma(r) = \text{Volatility}(\text{IntRate}(i))$ and $\mu(r) - \lambda(r)\sigma(r) = \text{RiskNeutralDrift}(\text{IntRate}(i))$.

This fragment of code is just the time stepping, above it would go declarations and setting up the payoff. Below it would go the outputting. It does not implement any boundary conditions at $i = 1$ or at $i = \text{NoIntRateSteps}$, these would depend on the contract being valued.

```
For i = 1 To NoIntRateSteps - 1
    If RiskNeutralDrift(IntRate(i)) > 0 Then
        Delta(i) = (Vold(i + 1) - Vold(i)) / dr
        RNDrift = RiskNeutralDrift(IntRate(i) + 0.5 * dr)
    Else
        Delta(i) = (Vold(i) - Vold(i - 1)) / dr
        RNDrift = RiskNeutralDrift(IntRate(i) - 0.5 * dr)
    End If
    Gamma(i) = (Vold(i + 1) - 2 * Vold(i) + Vold(i - 1)) / (dr * dr)
    VNew(i) = Vold(i) + Time step * (0.5 * Volatility(IntRate(i)) _
        * Volatility(IntRate(i)) * Gamma(i) + RNDrift _
        * Delta(i) - IntRate(i) * Vold(i))
Next i
```

³ That's 'wind' as in breeze, not as in to wrap or coil.

⁴ This choice won't make much difference to the result but it does help to make the numerical method 'conservative,' meaning that certain properties of the partial differential equation are retained by the solution of the difference equation. Having a conservative scheme is important in computational fluid dynamics applications, otherwise the scheme will exhibit mass 'leakage.'

To get back the $O(\delta S^2)$ accuracy of the central difference with a one-sided difference you can use the approximations described in section 28.5.

We have seen that the explicit finite-difference method suffers from restrictions in the size of the grid steps. The explicit method is similar in principle to the binomial tree method, and the restrictions can be interpreted in terms of risk-neutral probabilities. The terms A , B and C are related to the risk-neutral probabilities of reaching the points $i - 1$, i and $i + 1$. Instability is equivalent to one of these being a negative quantity, and we can't allow negative probabilities. More sophisticated numerical methods exist that do not suffer from such restrictions.

The advantages of the explicit method

- It is very easy to program and hard to make mistakes.
- When it does go unstable it is usually obvious.
- It copes well with coefficients that are asset and/or time dependent.
- It is easy to incorporate accurate one-sided differences.

The disadvantage of the explicit method

- There are restrictions on the time step so the method can be slower than other schemes.

28.16 **SUMMARY**

The diffusion equation has been around for a *long* time. Numerical schemes for the solution of the diffusion equation have been around quite a while too, not as long as the equation itself but certainly a lot longer than the Black–Scholes equation and the binomial method. This means that there is a great deal of academic literature on the efficient solution of parabolic equations in general.

FURTHER READING

- For general numerical methods see Johnson & Riess (1982) and Gerald & Wheatley (1992).
- The first use of finite-difference methods in finance was due to Brennan & Schwartz (1978). For its application in interest rate modeling see Hull & White (1990b).
- An excellent, well-written, book on numerical methods in finance is Ahmad (2008).
- See *PWOQF2* for more on finite-difference methods including techniques for higher dimensions.

Time Out...

Over and over again

Once you're happy with writing explicit finite-difference code you'll find yourself using it for many pricing problems. In fact, you may even use the same piece of core code for many types of instruments, it's that robust. But even writing the code from scratch will become second nature. Eventually, expect to take no more than 10 minutes to write basic pricing code *with no mistakes*.



EXERCISES

1. Write a program to value European call and put options by solving the Black–Scholes equation with suitable final and boundary conditions. Include a constant, continuous dividend yield on the underlying share. Output option value, delta, gamma and theta.
2. Adjust your program to value call options with the forward price as underlying.
3. Write a program to value a down-and-out call option, with barrier below the strike price.
4. Write a program to value compound options of the following form:
 - (a) call on a call;
 - (b) call on a put;
 - (c) put on a call;
 - (d) put on a put.
5. Alter your compound option program to value a chooser option which allows you to buy a call or a put at expiry.

